Living State Machine

Gated, Ablatable Memory for Long-Horizon Preference Recall
Around a Frozen Language Model

Kalen Patterson
Independent research
kcpatterson27@gmail.com

October 2026

Abstract

Long-horizon assistants need memory that is cheap, inspectable, and resistant to drift. This report describes **Living State Machine (LSM)**, a local-first wrapper around a frozen language model that combines a compact dual state, tiered episodic memory, habituation, Hebbian association, gated writes, a deterministic verifier, and surprise-modulated plasticity. Every mechanism is ablatable through configuration. On a matched-budget preference-recall benchmark, LSM scored **1.000** against **0.100** for naive retrieval-augmented generation across three seeds; a harder decoy-heavy stress benchmark was treated as a stress test rather than a success metric. The gated writer reached an F1 of **0.98** and was necessary under decoy load, where always-store collapsed to zero. The report also documents three substantive negative results—a multi-timescale memory variant, an encoder verifier, and a Stentor-style recurrent core—plus one evaluation-harness fault that initially looked like a model regression. The central claim is deliberately narrow: under a matched context budget, structured, gated memory can outperform naive retrieval. It is not a claim that the wrapper beats a full-context model on every long transcript.

Keywords: long-horizon memory; retrieval-augmented generation; gated writes; state-space models; local-first AI; evaluation hygiene

1 Introduction

A frozen language model is stateless between calls. The common workarounds are prompt stuffing, retrieval-augmented generation, or a separate memory service. Each buys context at a cost: full history grows without bound, retrieved snippets are brittle under decoys, and an LLM-written summary can quietly turn an assistant's guess into a persistent fact. On consumer hardware the constraint is sharper: a 12 GB inference budget leaves little room for a second model to manage memory.

LSM studies a narrower question: can a compact set of inspectable, biologically inspired rules around a frozen model improve long-horizon preference recall at a *matched context budget*? The system is a wrapper, not a fine-tune. It keeps a small working state s_t, a durable episodic store, and a per-turn control loop. The research track closed in July 2026; this report is a technical account of the evidence, including what failed.

The contributions are:

- A wrapper architecture in which every memory mechanism has a `Null*` implementation, so ablations are configuration changes rather than rewrites.

- A deterministic facts-backed projection that replaces LLM-authored state after paraphrase errors were observed in live sessions.
- A gated writer and verifier layer, plus evidence that gating is what survives a decoy-heavy stress bench.
- A reproducible benchmark suite and a deliberately honest negative-results record, including a diagnosis of a false regression caused by one embedder choice.

2 Related Work

Retrieval-augmented generation (RAG) conditions a model on retrieved passages rather than the full corpus [1]. It is effective when retrieval is clean, but it assumes the retrieved unit is useful in isolation. LSM instead retrieves from an episodic memory that has been filtered by a writer gate and scored by several signals.

Memory-augmented agents such as MemGPT treat the context window as a managed resource and move information between working and external memory [2]. Generative Agents maintain a memory stream with reflection and retrieval [3]. LSM shares the tiered-memory premise but differs in two ways: the language model is frozen and local, and the persistent state is rendered deterministically from structured facts rather than authored by the model.

Long-context transformers reduce the need for retrieval by extending attention [4], while state-space models offer linear-time sequence mixing [5]. LSM's Phase 2 explored a small state-space core, but the production path remained a wrapper; the trained core had to clear a no-regression gate rather than replace the baseline. Long context is not free: accuracy degrades when the relevant passage sits in the middle of a long prompt [6], which is the practical argument for retrieving a small, scored set rather than forwarding a transcript.

Recent work evaluates assistant memory more broadly than this report does. LongMemEval measures information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention over long interactive histories [7]; LSM's suite is narrower and matched-budget, but it names the same failure modes and includes a decoy-heavy stress task rather than hiding behind the headline. HippoRAG builds a knowledge graph over memories and retrieves with personalised PageRank [8]; LSM instead keeps a compact dual state plus an episodic log and deliberately does not build a graph. Managed memory layers such as Mem0 extract and consolidate facts behind a hosted API [9]; LSM is local-first and renders its state deterministically from structured facts, trading breadth for inspectability.

Habituation and associative learning are classic biological mechanisms [10], and regularization methods for sequential learning address a related stability problem [11]. LSM borrows the vocabulary—habituation, association, surprise, plasticity—but evaluates each mechanism as an engineering component, not as a biological claim.

3 System

3.1 Per-turn loop

Each turn follows a fixed control loop: embed the turn; score memory candidates with cosine similarity, habituation damping, association boost, and trust; retrieve the top k; update the working state; compose a prompt from the state projection and retrieved snippets; call the frozen model;

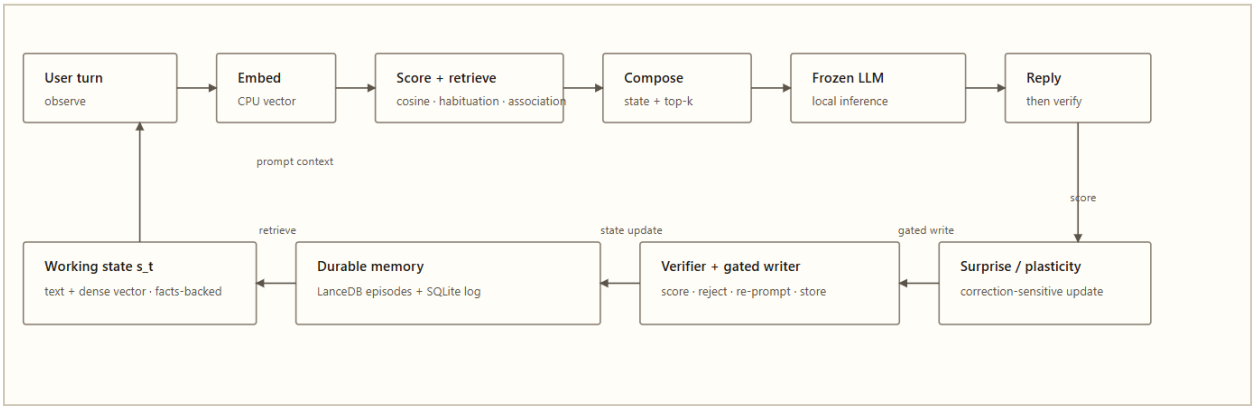


Figure 1: The LSM control loop. The durable store is read through the scored retriever and written only through the verifier and gate. The working state is a compact projection, not a transcript.

score the outcome; run the verifier; write only what passes the gate; and update plasticity signals. The loop is deliberately small enough to inspect in logs.

3.2 State and memory tiers

s_t is a dual representation: a short text projection injected into the prompt (roughly 400–600 characters) and a dense vector used for retrieval conditioning. Durable memory holds verbatim atoms in LanceDB plus an episodic SQLite log. Fast, session, and durable tiers are connected by promotion, hit-count, vitality decay, and a background consolidator. The mental model is explicit: the state is a compressed, inspectable summary; durable memory holds the evidence.

3.3 Mechanisms

Habituation damps repeated low-value input; association boosts memories that co-occur with the current turn; trust carries outcome history; surprise-modulated plasticity amplifies the learning rate on correction-like turns. The facts-backed updater renders `s_t.text` deterministically from structured facts with recency, correction-aware slot ordering, and budget trimming. The LLM-authored summarizer remains only as an ablation. This change followed a live failure in which a vocative was absorbed into state as a game title and then inherited by every later turn.

3.4 Gated writing and verification

The verifier scores candidates rather than authoring text. Its five weighted components cover fact contradiction, identity drift, correction violation, list-merge errors, and length overflow. The score gates three surfaces: the writer (rejected text never becomes durable memory), the reply (one re-prompt with fix hints, then the lower-energy draft), and the legacy summarizer. The verifier is deterministic by default; a linear/hybrid learned head was evaluated, while an encoder variant was logged as a negative.

3.5 Ablation as dependency injection

Every mechanism sits behind a small Protocol with a `Null*` implementation. The eight benchmark configurations, the verifier family, and the state-updater family are therefore configuration flips.

A run copies its YAML into the run directory with `metrics.json`, raw outputs, and logs; a run without its config is not treated as a run.

4 Experimental Setup

The evaluation suite contains eight tasks: matched-budget preference recall (`recall_pref_v1`), a harder decoy/long-gap stress task (`recall_pref_v2`), habituation and association signal benches, memory-write precision, multi-session consistency, fact correction, and verifier calibration. Headline claims use seeds 0, 1, and 2; Phase 2 integration checks are seed 0 only and are labelled as such.

The primary baselines are naive RAG and a raw LLM with a matched context budget. A full-transcript raw LLM is reported separately because it is not matched: it sees the entire history and therefore answers a different resource question. Hardware was a single RTX 5060 Ti (16 GB VRAM, 32 GB RAM) running a frozen local model through Ollama. A one-time cloud burst on a RunPod L40S cost approximately **$8.04** and consumed about 9.3 hours for QLoRA and small-core training. The local test suite passes 273 tests at submission.

Two implementation details matter when reading the tables. Every end-to-end run uses the local hash embedder (384 dimensions) and the legacy `LLMSummarizerUpdater`, not the production `FactsBackedUpdater`; the tables therefore measure retrieval, state, and writer ablations rather than daily-chat defaults. `recall_pref_v1` uses 10 items and 20 distractor turns per seed; `recall_pref_v2` uses 15 items and 45 distractor turns. Unless a row says otherwise, Tables 1 and 2 report seeds 0–2; the Phase 2 integration checks and the component benches in Table 3 are single-seed or single-dataset and are labelled where used.

Reporting. Each end-to-end cell is the mean and standard deviation across seeds 0–2, with 10 prompts per seed on v1 and 15 on v2; the dispersion is across seeds, not across prompts. No confidence intervals are reported and no multiple-comparison correction is applied. The component benches in Table 3 are single-seed (writer, 60 signal / 60 filler turns) or single-dataset (verifier, $n = 13$) and are directional.

Leakage controls. Prompts are generated per seed from a fixed template; the preference value is inserted at run time and is absent from the distractor pool. No preprocessing is fitted on the evaluation split, and every system in a table sees the same prompt set. The v2 decoys are drawn from the same topic as the target preference, so topic matching alone cannot solve the task.

Baseline fairness. All systems share the frozen model, prompt template, and scoring script; the matched-budget baselines receive the same context budget as LSM. Baselines are not tuned per task beyond the shared top-k and score threshold.

Negative control. Re-scoring the committed outputs with the correct labels shuffled across items drops v1 accuracy from 1.000 to 0.000 and v2 from 0.089 to 0.022. The scorer is therefore sensitive to the stated preference rather than passing answers for an unrelated reason.

Validity check. Running the same 10-item configuration with memory disabled scores 0.000 (seed 0, $n = 10$), against 1.000 with memory enabled. The benchmark therefore depends on the memory path rather than on the model's prior alone.

Table 1: Preference recall (accuracy; seeds 0–2). v2 is a stress bench, not the headline success metric.

System	`recall_pref_v1`	`recall_pref_v2`
Naive RAG	0.100 ± 0.100	0.044 ± 0.038
Raw LLM, matched budget	0.000 ± 0.000	0.000 ± 0.000
Raw LLM, full transcript	1.000 ± 0.000	0.911 ± 0.038
LSM full	**1.000 ± 0.000**	0.089 ± 0.038

Table 2: Ablations on `recall_pref_v1` (seeds 0–2 means). Δ is relative to LSM full.

Ablation	Accuracy	Δ	Reading
LSM full	1.000 ± 0.000	—	Ceiling
No habituation	0.967 ± 0.058	−0.033	Measured contribution
No writer gate	0.933 ± 0.115	−0.067	Writer matters
Flat memory	0.933 ± 0.058	−0.067	Tiering matters
No association	1.000 ± 0.000	0.000	Bench too easy
Multi-memory	1.000 ± 0.000	0.000	No separation

5 Results

On the matched-budget headline task, LSM reached the ceiling while naive RAG remained at 0.100. The full-context baseline also reached 1.000 on v1, so the result is not evidence that LSM beats full context on that task. On the harder v2 stress bench, the full-context model remained strong at 0.911 while LSM fell to 0.089; the matched raw baseline was 0.000. The honest reading is that v2 measures the value of full-history access, not a wrapper failure, and it is reported to keep the headline claim in bounds.

The Phase 2 integration gate held the headline benchmark at 1.000 when the trained hybrid SSM updater was projected into s_t, and both the Transformers base and the QLoRA adapter reached 1.000 after the harness fix described below. The trained core was therefore accepted as an optional state updater, not as a replacement for the wrapper. On the decoy-heavy v2 bench the always-store arm scored 0.000 against 0.089 for the rules writer (run `20260530-072506`); the v1 ceiling alone would not show that gating matters.

6 Negative Results and Failure Analysis

Multi-timescale memory did not separate. A multi-timescale memory variant produced a small negative effect ($\Delta \approx -0.05$). The result was kept rather than tuned into the headline: on this bench, additional tiers added complexity without a measurable gain.

The encoder verifier collapsed. The MiniLM cross-encoder verifier scored 0.211 macro-F1 at $n = 13$, effectively rejecting everything. With a small labeled set, the encoder had no room to learn the decision boundary. The linear and hybrid heads were retained as directional evidence; the encoder result is a named negative, not a hidden hyperparameter search.

Stentor-style recurrence lost to a minimal SSM. The ambitious Phase 2 hypothesis—that a Stentor-inspired recurrent core would beat a minimal diagonal SSM on language-modeling loss—failed: final loss was 5.21 versus 2.95 on TinyStories. The comparison was not clean: step counts

Table 3: Component evidence: writer precision, retrieval MRR, and verifier calibration.

Component	Result	Note
Writer — always	F1 0.67	Precision 0.50, FPR 1.00
Writer — rules	F1 0.98	Precision 1.00, recall 0.97, FPR 0.00
Writer — learned	F1 1.00	60 signal / 60 filler turns
Habituation signal	+0.068 MRR	$0.314 \rightarrow 0.382$
Association signal	+0.317 MRR	$0.454 \rightarrow 0.772$
Verifier — rule	0.556 macro-F1	Default V.1
Verifier — linear/hybrid	0.771 macro-F1	$n = 13$; directional only
Verifier — encoder	0.211 macro-F1	Reject-all collapse; negative

and sequence lengths were unequal, and the association-trace update was sequential. The minimal core was promoted because it passed the no-regression gate, and the negative was logged with its confounds. The honest next experiment is a parallelized association scan at matched compute.

A false regression caused by the evaluation harness. After QLoRA training, the adapter path appeared broken: base 0.100 and +LoRA 0.200. The model was not the problem. A hash embedder combined with a minimum score of 0.15 filtered out the preference line, returning zero memories; switching the eval embedder to fastembed restored both paths to 1.000. The correction was written into the phase report. The lesson is now a first-class check: inspect retrieved-item counts before interpreting an end-to-end score.

7 Limitations and Threats to Validity

- **Small, synthetic benchmarks.** The preference tasks are controlled and synthetic. They test a narrow memory behavior, not general assistant quality.
- **No external users.** Daily utility is a qualitative single-user observation; there is no retention or satisfaction study.
- **Seed asymmetry.** Headline results use three seeds; Phase 2 integration checks are seed 0 only. Treat those as directional.
- **Decoding variance.** The harness does not pin sampling temperature. Seed variation changes prompts, not decoding, so the reported dispersion mixes prompt and sampling variance.
- **Small verifier labels.** The learned/hybrid verifier result is $n = 13$ and is not paper-grade. The encoder negative is similarly small.
- **Matched-budget scope.** The headline claim is about naive RAG at a matched budget. Full-context raw LLMs remain stronger on the harder stress bench.
- **Hardware and cost.** Results come from one consumer GPU and one approximately $8 cloud burst; no multi-hardware replication.
- **Privacy boundary.** The system is local by default and supports optional encryption at rest, but the encryption path has not been independently audited.

Data and code availability. The implementation, configs, and the evaluation artifacts behind Tables 1–3 are in the repository; `paper/REPRODUCE.md` maps every row to its run directory.

8 Conclusion

LSM shows that a compact, inspectable memory wrapper can produce a large gain over naive RAG at a matched context budget, and that the writer gate is the component most likely to matter when memory is under decoy pressure. The result is deliberately bounded: it does not claim to beat full-context models, and its negative results are part of the contribution. The next steps are matched-compute recurrence experiments, a larger verifier corpus, multi-seed Phase 2 integration, and a public, reproducible release of the code and run artifacts.

Appendix: Red-Team Questions

The benchmark is synthetic and easy. True, and the paper says so. The v1 ceiling is a sanity check, not a general capability claim; the v2 stress bench and the negative results are where the evidence is bounded.

The hash embedder is not the production path. Correct. The tables measure the wrapper's retrieval, state, and writer ablations under a fixed embedder; the fastembed correction is reported separately, and the production path is not claimed to be identical.

The writer gate is just a filter. The gate is a filter, and the ablation shows what happens without it: 0.933 on v1 and 0.000 on the decoy-heavy v2 bench. The claim is about the gate's effect, not its novelty.

AI Assistance

AI tools assisted with code generation, drafting, and editorial review. The author designed the experiments, ran every evaluation, reviewed and tested all generated code (273 tests), re-derived every table from the committed run artifacts, and verified every citation against the published source. The author takes full responsibility for the content.

Code and Artifacts

The implementation, run scripts, and evaluation artifacts are in the author's repository, which will be released with the paper; `paper/REPRODUCE.md` maps each table row to its run directory. The tables were reconciled against the Phase 1 and Phase 2 reports on 2026-10-20 and against the raw experiment artifacts on 2026-10-27.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020. arXiv:2005.11401.

[2] C. Packer et al., "MemGPT: Towards LLMs as Operating Systems," 2023. arXiv:2310.08560.

[3] J. S. Park et al., "Generative Agents: Interactive Simulacra of Human Behavior," *UIST*, 2023. arXiv:2304.03442.

[4] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," 2020. arXiv:2004.05150.

[5] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," 2023. arXiv:2312.00752.

[6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the Middle: How Language Models Use Long Contexts," *TACL*, 12:157–173, 2024. arXiv:2307.03172.

[7] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," *ICLR*, 2025. arXiv:2410.10813.

[8] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models," *NeurIPS*, 2024. arXiv:2405.14831.

[9] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," 2025. arXiv:2504.19413.

[10] R. F. Thompson and W. A. Spencer, "Habituation: A Model Phenomenon for the Study of Neuronal Substrates of Behavior," *Psychological Review*, 73(1):16–43, 1966.

[11] J. Kirkpatrick et al., "Overcoming Catastrophic Forgetting in Neural Networks," *PNAS*, 114(13):3521–3526, 2017.